

Replit Agent Prompt — Update SignTakeoff App

Context

I have an existing Node.js/TypeScript SignTakeoff app that currently sends full plan PDFs to the Gemini API for sign takeoff analysis. I want to restructure it to use a cost-efficient pipeline where most work is done in deterministic code and only 2-3 targeted Gemini API calls are made per takeoff. This will reduce cost from ~\$4-6 per takeoff to ~\$0.01-0.03.

Current state

- Node.js/TypeScript app
- Gemini API backend (keep this — do not switch to a different LLM)
- Full workflow: upload PDF → takeoff → marked-up PDF → export
- Currently sends many/all plan pages to Gemini (expensive)

What to change — the pipeline architecture

Restructure the app into a 7-step pipeline. Steps 3 and 4b call Gemini. All other steps run as deterministic Node.js code with NO LLM calls.

Step 1 — PDF Intake (deterministic)

Use `pdf-lib` or `pdf-parse` (npm) to extract:

- Page count
- Metadata (title, author, creation date)
- Text from page 1 only (the drawing index)

Step 2 — Sheet Filtering (deterministic)

Parse the drawing index text from Step 1 to identify signage-relevant sheets. Search for these keywords in sheet titles:

```
const SIGNAGE_KEYWORDS = ['signage', 'plaque', 'sign schedule', 'sign details', 'millwork'];
```

```
const FLOOR_PLAN_KEYWORDS = ['floor plan', 'construction plan'];
```

```
const EGRESS_KEYWORDS = ['egress', 'life safety', 'occupant load'];
```

```
const CODE_KEYWORDS = ['mounting heights', 'accessibility', 'code review'];
```

Map each keyword hit to its sheet ID and PDF page number. Result: a list of ~8-12 relevant page numbers out of 100-169 total.

Step 3 — Plaque Schedule Extraction (GEMINI API CALL #1)

Rasterize ONLY the signage schedule page (1 page) at 200 DPI using `pdf-poppler` or `sharp` + `pdf2pic` npm packages.

Send this single page image to Gemini with this prompt:

You are reading an architectural signage/plaque schedule sheet. Extract every plaque type shown. For each, return JSON:

```
{  
  
  "plaques": [  
  
    {  
  
      "type_id": "A",  
  
      "name": "Room Name Sign",  
  
      "braille": true,  
  
      "insert": false,  
  
      "insert_size": null,  
  
      "letter_height": "5/8" cap",  
  
      "trigger": "Default room ID for occupied rooms",  
  
      "maps_to_column": "Room ID"  
  
    },  
  
  ],  
  
  "general_notes": {
```

```

    "code_citation": "521 CMR 41 MAAB",

    "mounting_height_sheet": "A-000",

    "fallback_mounting": "nearest adjacent wall if no latch-side space"

  }

}

```

Return ONLY valid JSON, no explanation.

Cost: ~2,000 tokens input + ~500 tokens output.

Step 4a — Room Label Extraction (deterministic)

This is the most important deterministic step. Use [pdf.js](#) (pdfjs-dist npm) or [pdf-parse](#) to extract all text with positions from each floor plan page.

```

interface RoomLabel {

  roomNumber: string;    // e.g., "109"

  roomName: string;      // e.g., "POLICE CHIEF"

  level: string;         // e.g., "LEVEL 1"

  sheetId: string;       // e.g., "A-101"

  pdfPage: number;       // e.g., 25

  coords: { x: number; y: number }; // normalized 0-1000

  bbox: { x0: number; y0: number; x1: number; y1: number };

}

```

Algorithm:

1. Get all text items with positions from the PDF page
2. Find items matching room number pattern: [/^\[1-9\]\d{2}\[A-Z\]?\\$/](#)
3. For each number, look for CAPS text directly above (within ~40pt vertical, ~100pt horizontal)

4. Use fuzzy matching for abbreviations:

```
const SYNONYMS: Record<string, string[]> = {  
  
  'VEST': ['VEST', 'VESTIBULE'],  
  
  'EMERG': ['EMERG', 'EMERGENCY'],  
  
  'INTERROG': ['INTERROG', 'INTERROGATION'],  
  
  'CORR': ['CORR', 'CORRIDOR'],  
  
};
```

5. Filter out numbers in the legend/notes area (typically right 20% of page)
6. Normalize coordinates to 0-1000 grid:

```
const normX = Math.round(((pdfX - planLeft) / planWidth) * 1000);  
  
const normY = Math.round((1 - (pdfY - planBottom) / planHeight) * 1000); // flip Y for  
bottom-left origin
```

Step 4b — Occupant Loads Table (GEMINI API CALL #2)

Rasterize the egress/occupant loads page (1 page) at 200 DPI.

Send to Gemini with:

You are reading an architectural egress drawing with an Occupant Loads table.

Extract the occupant load and occupancy group for every room listed.

Return ONLY valid JSON:

```
{  
  
  "rooms": [  
  
    {"room_num": "138", "room_name": "TRAINING/COMMUNITY/EOC",  
  
      "occupant_load": 240, "occupancy_group": "A-2"}  
  
  ]  
}
```

```
}
```

Cost: ~2,000 tokens input + ~1,000 tokens output.

Step 4c — Build Room Inventory (deterministic)

Merge the room labels from 4a with the occupant loads from 4b. Derive flags for each room:

```
interface RoomInventory extends RoomLabel {  
  
    occupantLoad: number | null;  
  
    occupancyGroup: string | null;  
  
    flags: {  
  
        isRestroom: boolean;    // TOILET, BATH, SHOWER, WC, RESTROOM, LAVATORY  
  
        isStair: boolean;       // starts with STAIR  
  
        isElevator: boolean;    // ELEV, ELEVATOR  
  
        isVestibule: boolean;   // VEST, VESTIBULE  
  
        isCorridorOrHall: boolean; // HALL, CORR, CORRIDOR, LOBBY, FOYER  
  
        isVehicleBay: boolean; // APPARATUS, VEHICLE BAY, SALLY PORT, GARAGE  
  
        isMepUnoccupied: boolean; // MECHANICAL, ELECTRICAL, etc. AND occupantLoad == 0  
  
        isVariableUse: boolean; // TRAINING, COMMUNITY, EOC, MULTIPURPOSE  
  
        isPublicFacing: boolean; // LOBBY, RECEPTION, WAITING  
  
        isStaffOnly: boolean; // "STAFF" in name  
  
        isAssembly: boolean; // occupancyGroup in ['A-2', 'A-3'] OR occupantLoad >= 50  
  
    };  
}
```

Step 5 — Apply Rules R1-R15 (deterministic)

This is pure TypeScript logic. No LLM call needed.

```
function applyRules(room: RoomInventory, plaqueSchedule: PlaqueSchedule): TakeoffEntry[] {

  const signs: TakeoffEntry[] = [];

  // R1: Default Room ID

  if (!room.flags.isCorridorOrHall && !room.flags.isVehicleBay &&
    !room.flags.isMepUnoccupied && !room.flags.isStair && !room.flags.isElevator) {

    // R2: Variable use → insert variant

    if (room.flags.isVariableUse) {

      const qty = room.flags.isAssembly ? 2 : 1; // dual function = 2

      signs.push({ signType: 'Room ID w/insert', qty, ruleRef: 'R2',

        color: '#7c3aed', plaqueTypeId: findPlaque('insert', plaqueSchedule) });

    } else {

      signs.push({ signType: 'Room ID', qty: 1, ruleRef: 'R1',

        color: '#2563eb', plaqueTypeId: 'A' });

    }

  }

  // R8: Restroom
```

```
if (room.flags.isRestroom) {
```

```
    const variant = selectRestroomVariant(room);
```

```
    signs.push({ signType: 'Restroom', qty: 1, ruleRef: 'R8',
```

```
        color: '#14b8a6', plaqueTypeId: variant });
```

```
}
```

```
// R9: Exit
```

```
if (room.flags.isVestibule ||
```

```
    (room.flags.isPublicFacing && room.roomName.includes('LOBBY')) {
```

```
    signs.push({ signType: 'Exit', qty: 1, ruleRef: 'R9', color: '#dc2626' });
```

```
}
```

```
if (room.flags.isAssembly && (room.occupantLoad ?? 0) >= 50) {
```

```
    signs.push({ signType: 'Exit', qty: 2, ruleRef: 'R9', color: '#dc2626' });
```

```
}
```

```
// R10: Capacity
```

```
if (room.flags.isAssembly) {
```

```
    signs.push({ signType: 'Max Occupancy', qty: 1, ruleRef: 'R10', color: '#f59e0b' });
```

```
}
```

```
// R14: Office Directory (corridors/halls only)
```

```
if (room.flags.isCorridorOrHall && room.flags.isPublicFacing) {
```

```
    signs.push({ signType: 'Office Directory', qty: 1, ruleRef: 'R14', color: '#3b82f6' });  
  
  }  
  
  return signs;  
  
}
```

Handle stairs and elevators separately (not per-room):

```
function applyStairRules(stairs: StairEntity[], levels: string[]): TakeoffEntry[] { ... }
```

```
function applyElevatorRules(elevators: ElevatorEntity[]): TakeoffEntry[] { ... }
```

Step 6 — Verification (deterministic)

Run 10 checks, return pass/fail for each:

```
interface ValidationResult {  
  
  roomCoverage: 'PASS' | 'FAIL';  
  
  restroomCount: 'PASS' | 'FAIL';  
  
  exitFloorCheck: 'PASS' | 'FAIL';  
  
  stairMath: 'PASS' | 'FAIL';  
  
  elevatorCount: 'PASS' | 'FAIL';  
  
  assemblyCapacity: 'PASS' | 'FAIL';  
  
  zeroSignCheck: 'PASS' | 'FAIL';  
  
  doorToSignRatio: 'PASS' | 'FAIL';  
  
  assemblyThreshold: 'PASS' | 'FAIL';  
  
  subtotalMath: 'PASS' | 'FAIL';  
  
}
```


}

Step 7 — Generate Outputs (deterministic)

7a. JSON response (for the frontend): Return the full structured JSON matching the schema from the merged system prompt.

7b. Marked-up PDF using `pdf-lib` (npm):

1. Load the original floor plan page from the PDF
2. For each room with assigned signs:
 - Draw a small circle offset ~30pt right of the room label's bbox
 - Draw a dashed leader line from the label edge to the marker
 - Use gold fill (`#fbbf24`) for normal rooms, red (`#ef4444`) for missed rooms
 - Stack color-coded sign-type pills next to each marker
3. Add a legend in the upper-left corner
4. Save as a new PDF

7c. Excel export using `exceljs` (npm): Generate the takeoff spreadsheet matching the customer column format.

Step 3 fallback — Optional GEMINI API CALL #3

If any room's rule application is ambiguous (e.g., unclear whether it's public-facing, unclear occupancy classification), batch those edge cases into a single Gemini call:

Given these rooms with unclear attributes, determine the correct classification:

[list of ambiguous rooms with context]

Return JSON with your determination for each.

This call only fires when there ARE ambiguities. Most takeoffs won't need it.

NPM packages needed

- `pdfjs-dist` or `pdf-parse` — PDF text extraction with positions
- `pdf2pic` or `pdf-poppler` — PDF page rasterization (for the 2 Gemini calls)
- `sharp` — image processing (resize rasterized pages before sending to Gemini)
- `pdf-lib` — PDF manipulation (for marked-up PDF generation)
- `exceljs` — Excel file generation
- `@google/generative-ai` — Gemini API (already installed)

File structure for the pipeline

src/

pipeline/

intake.ts // Step 1: PDF metadata extraction

sheetFilter.ts // Step 2: Drawing index parsing + sheet identification

plaqueSchedule.ts // Step 3: Gemini call to read signage sheet

roomExtraction.ts // Step 4a: Programmatic room label extraction

occupantLoads.ts // Step 4b: Gemini call to read egress sheet

roomInventory.ts // Step 4c: Merge rooms + loads + derive flags

rulesEngine.ts // Step 5: R1-R15 rule application (pure logic)

verification.ts // Step 6: 10 validation checks

outputJson.ts // Step 7a: Structured JSON assembly

markedUpPdf.ts // Step 7b: Annotated plan PDF generation

excelExport.ts // Step 7c: Spreadsheet generation

rules/

roomIdRules.ts // R1-R7

restroomRules.ts // R8

exitRules.ts // R9

capacityRules.ts // R10

stairRules.ts // R11

elevatorRules.ts // R12

evacMapRules.ts // R13

directoryRules.ts // R14

exclusionRules.ts // R15

types/

takeoff.ts // All TypeScript interfaces

utils/

synonyms.ts // Room name fuzzy matching

coordinates.ts // PDF → normalized coordinate conversion

pdfHelpers.ts // Rasterization, text extraction utilities

Key principle: the Gemini system prompt is SHORT

Do NOT put the full rulebook in the Gemini system prompt. The rules engine runs in TypeScript. Gemini is only used for:

1. Reading the plaque schedule from a rasterized page (structured extraction)
2. Reading the occupant loads table from a rasterized page (structured extraction)
3. Optionally resolving ambiguous room classifications

Each Gemini call gets a focused, task-specific prompt (~200 words) not the full methodology document. The methodology lives in your TypeScript code.

Migration plan

1. First, add the pipeline modules alongside the existing Gemini-heavy flow
2. Route new takeoffs through the pipeline
3. Compare pipeline output against the old Gemini output for a few test projects
4. Once validated, remove the old Gemini-heavy flow
5. The old system prompt becomes a reference document, not an active prompt

Cost target

- Current: ~~270K tokens per takeoff~~ (\$4-6 at Gemini 1.5 Pro pricing)
- Target: ~~4-7K tokens per takeoff~~ (\$0.01-0.03)

- The 2 mandatory Gemini calls (plaque schedule + occupant loads) total ~4K input tokens
- The optional ambiguity call adds ~3K when needed
- All other processing is free (runs in Node.js)